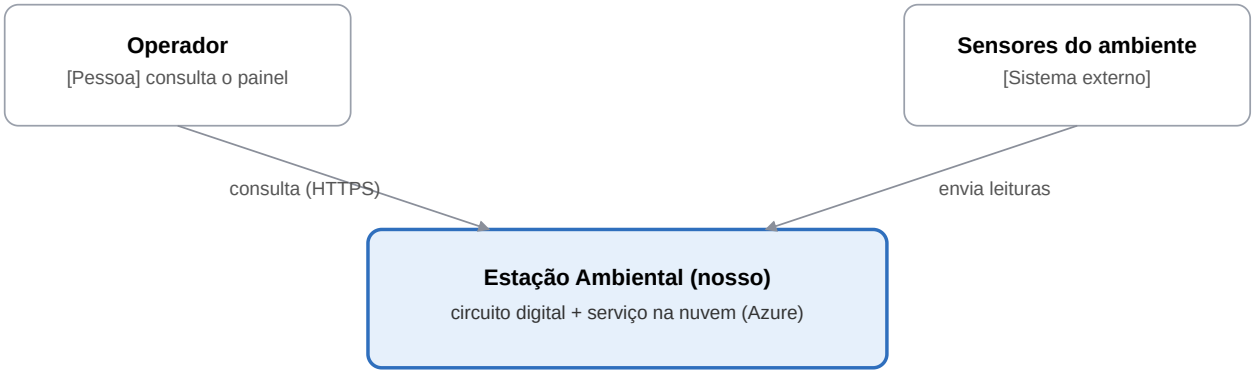
Arquitetura da Estação Ambiental (Modelo C4)

Projeto INTERFACE, ExpoTech 2026.2. Este documento descreve a arquitetura em três níveis do Modelo C4: contexto, contêineres e componentes. Tudo o que está descrito aqui **está implementado, publicado na Azure e testado** (versão de 28/10/2026).

Nível 1: Contexto do sistema

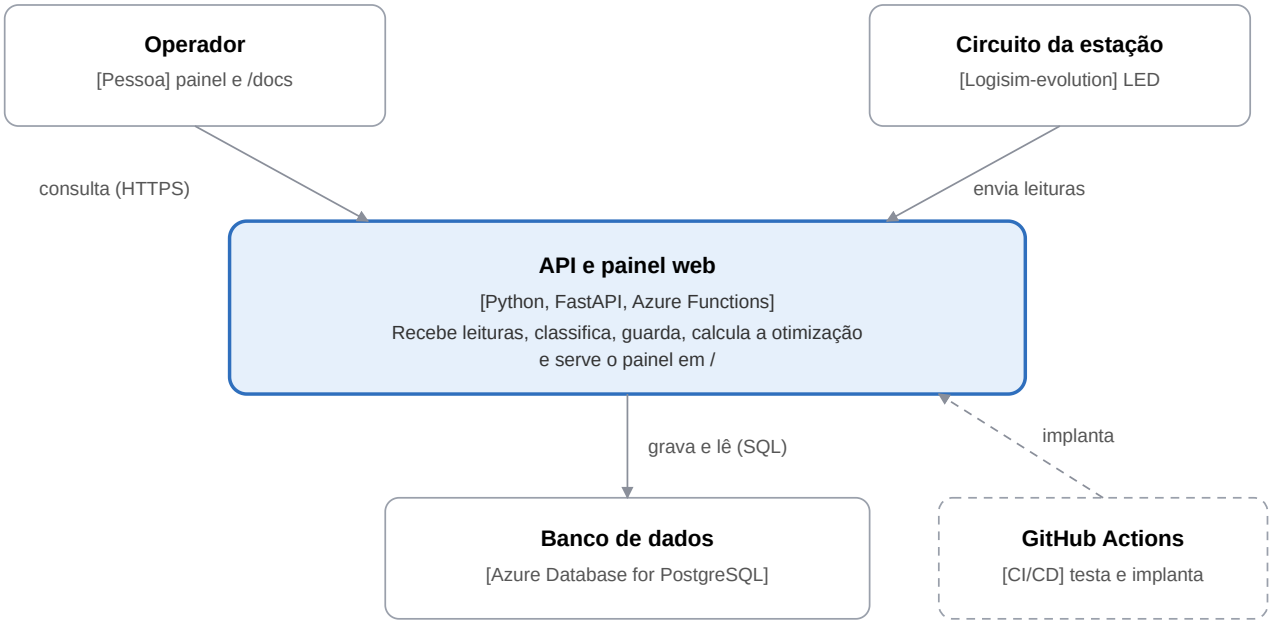
A Estação Ambiental mede temperatura, umidade e luminosidade de um ambiente, decide se está tudo normal, em atenção ou em alarme, guarda o histórico em um banco na nuvem e calcula com que frequência cada sensor deve ler para gastar menos energia.



Elemento	Tipo	Papel
Operador	Pessoa	Consulta o estado, o histórico e a frequência recomendada pelo painel web e pela interface da API
Sensores do ambiente	Sistema externo	Fornecem temperatura, umidade e luminosidade
Estação Ambiental (nosso)	Sistema	Circuito digital mais serviço em nuvem

Nível 2: Contêineres

O sistema se divide em cinco partes que rodam separadamente. O ponto de contato entre o circuito e o serviço em nuvem é o envio das leituras pela API (`integracao/enviar_casos.py`).

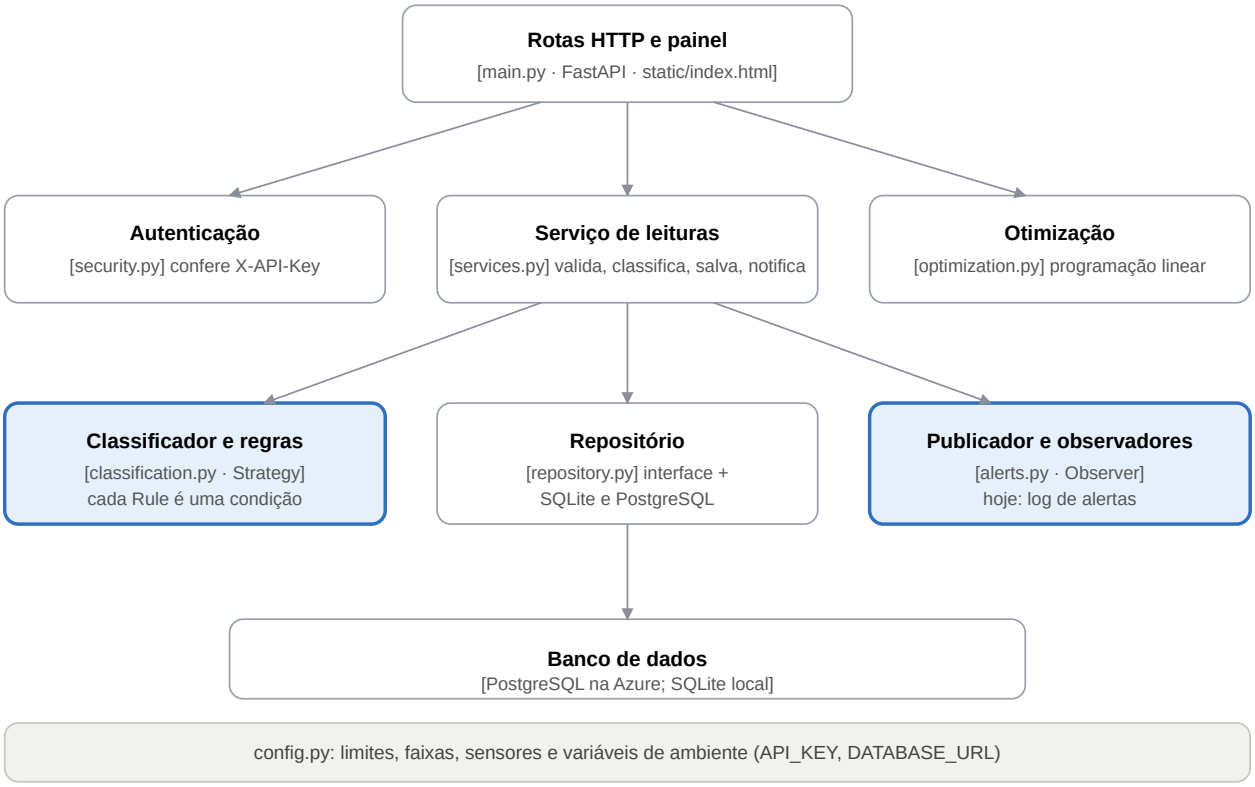


Contêiner	Tecnologia	Responsabilidade	Situação
Circuito da estação	Logisim-evolution	Compara cada leitura com os limites e acende o LED normal, atenção ou alarme	**Pronto (simulado)**
API e painel web	Python, FastAPI, Azure Functions (plano de consumo flexível)	Recebe leituras, classifica, guarda, calcula a otimização e serve o painel	**No ar na Azure**
Banco de dados	Azure Database for PostgreSQL (servidor flexível); SQLite só em desenvolvimento local	Guarda o histórico de leituras	**No ar na Azure**
Pipeline de entrega	GitHub Actions	Roda os testes (com PostgreSQL próprio) e publica a API a cada push na main	**Testes ativos; publicação ao cadastrar o segredo**
Monitoramento	Application Insights	Recebe os logs da API, inclusive os alertas	**Ativo**

Como os dados circulam: o circuito decide localmente qual LED acender. A API aplica a mesma regra (limites iguais aos do circuito) para registrar o estado no histórico. Na estação real, um microcontrolador enviaria cada leitura por HTTPS (método POST, com o cabeçalho `X-API-Key`). Na simulação, o script `enviar_casos.py` envia à API os mesmos 5 casos testados no circuito e confirma que o resultado é o mesmo (5 de 5).

Nível 3: Componentes da API

A API é organizada em camadas. Somente `main.py` conhece HTTP; as regras ficam em módulos independentes.



Os dois componentes em azul são os padrões GoF do projeto (Strategy e Observer).

Componente	Arquivo	Responsabilidade
Rotas HTTP e painel	`main.py`, `static/index.html`	Recebe as requisições, valida o formato, devolve JSON e serve o painel; é onde as peças são ligadas e onde se escolhe o banco (PostgreSQL se houver `DATABASE_URL`, senão SQLite)
Autenticação	`security.py`	Confere a chave `X-API-Key` com a variável de ambiente `API_KEY`
Serviço de leituras	`services.py`	Valida, classifica, salva e notifica (caso de uso)
Classificador e regras	`classification.py`	Padrão Strategy: cada regra é uma condição de alerta intercambiável
Publicador e observadores	`alerts.py`	Padrão Observer: avisa quem reage a leituras novas (hoje, o log de alertas)
Repositório	`repository.py`	Interface de persistência com duas implementações (SQLite e PostgreSQL) que passam no mesmo conjunto de testes
Otimização	`optimization.py`	Programação linear que define as leituras por hora de cada sensor
Configuração	`config.py`	Limites, faixas, parâmetros dos sensores e variáveis de ambiente

Interface da API

Endereço publicado: `https://estacao-api-87rj.azurewebsites.net` (painel em `/`, documentação interativa em `/docs`).

Método e rota	Autenticação	Função
`GET /`	Não	Painel web: estado atual, simulador, histórico e otimização
`GET /health`	Não	Verifica se a API está no ar
`POST /readings`	Cabeçalho `X-API-Key`	Registra uma leitura e devolve o estado (`desligado`, `normal`, `atencao` ou `alarme`)
`GET /readings?limit=50`	Não	Lista as últimas leituras, a mais recente primeiro
`GET /optimization/schedule?orcamento_mj_hora=1000`	Não	Devolve as leituras por hora e o intervalo de cada sensor

Recursos na Azure

Recurso	Nome	Tipo
Grupo de recursos	`rg-expotech`	Brazil South
Banco gerenciado	`estacao-db-87rj`	Azure Database for PostgreSQL, servidor flexível, B1ms
API	`estacao-api-87rj`	Azure Functions, plano de consumo flexível, Python 3.12
Armazenamento	`stestacao87rj`	Conta exigida pela Function App
Monitoramento	`estacao-api-87rj`	Application Insights

Decisões de arquitetura

1. **FastAPI e Python.** Cria a API com pouco código e gera sozinho a tela de testes (`/docs`).
2. **Camadas com repositório por interface.** O banco foi trocado de SQLite para PostgreSQL gerenciado da Azure sem alterar rotas nem regras; os testes do repositório rodam nos dois bancos.
3. **Chave de API na escrita.** Somente quem tem a chave grava leituras. A chave e a conexão do banco vêm de variáveis de ambiente da Function App e nunca ficam no código.
4. **Mesma regra no circuito e na API.** Os limites são iguais (temperatura acima de 30, umidade abaixo de 30, luminosidade acima de 80), e o script de integração comprova que os dois dão o mesmo resultado nos 5 casos.
5. **Azure Functions no lugar do App Service.** A assinatura de avaliação veio com cota zero para planos do App Service; o código é o mesmo, só muda a entrada (`function_app.py`).
6. **Logs como base de observabilidade.** O observador de alertas escreve no log, que chega ao Application Insights.

7. **CI/CD no GitHub Actions.** Os testes rodam a cada push, inclusive contra um PostgreSQL de verdade; a publicação na Azure roda quando o perfil de publicação está cadastrado como segredo.

O que continua simulado

- O microcontrolador real enviando leituras: a simulação do projeto substitui essa etapa pelo script `enviar_casos.py`.
- A parte analógica (sensor, conversor e resistores) é justificada por cálculo no documento de Circuito e Física, não simulada.